SMART PROJECT DOCUMENTATION

Login & Register Page Routings

Authentication Flow, Frontend Routing Structure, and Route Protection

Overview: Smart Project-க்கான Login மற்றும் Register பக்கங்களின் Routing architecture, Route Guards மற்றும் Frontend Implementation பற்றிய முழுமையான குறிப்புகள் கீழே வழங்கப்பட்டுள்ளன.

1. Introduction & Objectives

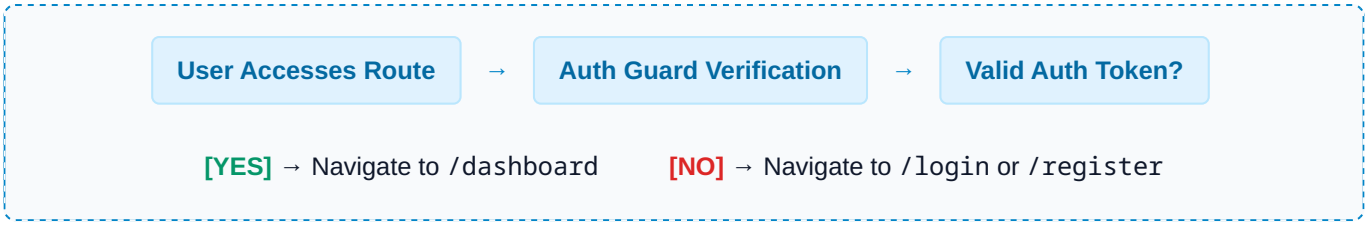
In modern web development (Smart Projects), page routing manages navigation between public pages (Login, Register, Forgot Password) and protected pages (Dashboard, Profile, Settings). Secure routing guarantees that unauthorized users cannot access application pages without authenticating first.

2. Routing Table Structure

The following routing map defines the main application routes:

Route Path	Component / Page	Access Guard	Redirection Rule
`/login`	LoginComponent	Public / Guest	If already logged in → redirect to `/dashboard`
`/register`	RegisterComponent	Public / Guest	If already logged in → redirect to `/dashboard`
`/forgot-password`	ForgotPasswordComponent	Public	None
`/dashboard`	DashboardComponent	Protected (AuthGuard)	If not logged in → redirect to `/login`
/ (Root)	Redirect Component	Public	Redirects automatically based on Auth status

3. Authentication Routing Flow



4. Implementation Code Structure

4.1 React Router Setup

Sample frontend routing implementation using React Router:

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import ProtectedRoute from './components/ProtectedRoute';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Public Routes */}
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />

        {/* Protected Smart Project Route */}
        <Route
          path="/dashboard"
          element={
            <ProtectedRoute>
              <Dashboard />
            </ProtectedRoute>
          }
        />

        {/* Fallback Route */}
        <Route path="*" element={<Navigate to="/login" replace />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

4.2 Protected Route Guard Logic

Prevents direct access to protected screens via manual browser URL entries:

```
import { Navigate } from 'react-router-dom';

const ProtectedRoute = ({ children }) => {
  const token = localStorage.getItem('authToken');

  if (!token) {
    return <Navigate to="/login" replace />;
  }

  return children;
};

export default ProtectedRoute;
```

5. Key Architecture Guidelines

- **Authentication State Storage:** Store JWT Tokens or Session Flags in `localStorage` or secure HTTP-only Cookies.
- **Automatic Redirection:** After a successful login on `/login` or account creation on `/register`, seamlessly route the user to `/dashboard`.
- **HTTP 401 Interceptors:** If an API returns an Unauthorized status code, clear session storage and immediately redirect the user to `/login`.
- **Guest Guard:** Prevent authenticated users from visiting `/login` or `/register` again without logging out.